

NOVSMM Enterprise — Auditoría Final de Certificación para Producción (Production Acceptance Audit)

Quiero que dejes de actuar como desarrollador del proyecto.

A partir de este momento debes actuar como un **Principal DevOps Engineer, Cloud Architect, Site Reliability Engineer (SRE)** y **Security Auditor** externo.

Asume que eres una empresa independiente contratada para certificar si NOVSMm puede entrar en producción.

No quiero optimizaciones nuevas.

No quiero nuevas funcionalidades.

No quiero refactors innecesarios.

Quiero una auditoría completamente imparcial y extremadamente crítica.

Si encuentras un problema, aunque sea pequeño, debes reportarlo.

No quiero respuestas optimistas.

Quiero encontrar absolutamente todos los riesgos antes de desplegar.

Objetivo

Realizar una auditoría completa de TODO el proyecto.

No debes asumir que algo funciona simplemente porque fue escrito.

Todo debe ser revisado nuevamente.

Debes desconfiar de todo.

Regla principal

No puedes dar por válido ningún archivo hasta haberlo inspeccionado completamente.

Cada script debe revisarse línea por línea.

Cada configuración debe validarse.

Cada flujo debe analizarse.

Si existe una posibilidad de fallo en producción, debe aparecer en el informe.

Quiero una auditoría de los siguientes componentes

1. Docker

Audita completamente:

- Dockerfile
- docker-compose.yml
- docker-compose.monitoring.yml
- imágenes utilizadas
- versiones
- usuarios root
- permisos
- capas
- tamaño de imagen
- variables
- healthchecks
- restart policies
- volumes
- networking
- bind mounts
- secrets
- build cache
- multi-stage build

Detecta:

- malas prácticas
 - imágenes inseguras
 - contenedores innecesarios
 - problemas de rendimiento
 - configuraciones incorrectas
-

2. Nginx

Audita:

- TLS
- HTTP2

- HTTP3
- gzip
- brotli
- headers
- CSP
- HSTS
- rate limiting
- reverse proxy
- websocket
- proxy buffers
- cache
- timeout
- keep alive
- uploads
- security headers

Indica cualquier configuración mejorable.

3. PostgreSQL

Revisar:

- índices
- relaciones
- foreign keys
- cascade
- transacciones
- isolation levels
- locks
- deadlocks
- VACUUM
- ANALYZE
- autovacuum
- tamaños
- secuencias
- migrations
- rollback

Buscar posibles race conditions.

Buscar consultas lentas.

Buscar tablas sin índices.

Buscar N+1 queries.

Buscar scans completos.

4. Redis

Revisar:

- cache
 - TTL
 - expiraciones
 - memory leaks
 - eviction policy
 - persistencia
 - AOF
 - RDB
 - reconexión
 - graceful degradation
 - pub/sub
 - BullMQ
 - colas
 - workers
-

5. Scripts Bash

Auditar absolutamente todos.

Incluyendo:

deploy.sh

backup.sh

restore.sh

healthcheck.sh

monitor-setup.sh

pre-deploy-check.sh

validate-postgres-redis.sh

smoke-test.sh

Todos deben revisarse línea por línea.

Buscar:

- rutas incorrectas
- comandos incompatibles
- errores silenciosos

- falta de validaciones
- permisos
- rollback
- manejo de errores
- logs
- códigos de salida
- compatibilidad Ubuntu 22.04 y 24.04

Todos deben usar buenas prácticas:

- set -Eeuo pipefail
 - trap
 - funciones reutilizables
 - mensajes claros
 - validación de dependencias
 - validación de variables
 - rollback cuando sea necesario
-

6. GitHub Actions

Revisar:

- CI
 - CD
 - secrets
 - cache
 - artefactos
 - permisos
 - runners
 - Docker build
 - seguridad
-

7. Seguridad

Realizar una auditoría completa.

Buscar:

- secretos expuestos
- variables inseguras
- credenciales hardcodeadas
- permisos excesivos
- SSRF
- XSS
- CSRF
- SQL Injection
- Path Traversal
- RCE

- XXE
- Insecure Headers
- JWT
- Cookies
- Session Fixation
- OAuth
- CORS
- Open Redirect

No asumir que porque existe protección significa que está correctamente implementada.

8. Rendimiento

Buscar:

- memory leaks
 - CPU spikes
 - cuellos de botella
 - consultas lentas
 - polling innecesario
 - duplicación de procesos
 - renderizados costosos
 - consumo de memoria
 - uso de Redis
 - uso de PostgreSQL
-

9. Observabilidad

Revisar:

Prometheus

Grafana

Alertmanager

Health endpoints

Logs

Sentry

Request IDs

Rotación de logs

Métricas faltantes

Alertas faltantes

10. Disaster Recovery

Validar:

Backups

Restore

Integridad

Rollback

Tiempo de recuperación

Pérdida máxima de datos

Automatización

11. Producción

Simular mentalmente un despliegue real y buscar cualquier escenario donde el sistema pueda fallar:

- caída de Redis
 - caída de PostgreSQL
 - caída del Worker
 - caída del servidor
 - falta de disco
 - memoria llena
 - reinicio inesperado
 - despliegue fallido
 - rollback
 - certificados vencidos
 - DNS
 - Cloudflare
 - webhooks
 - OAuth
 - SMTP
-

No quiero suposiciones

Si algo no puede verificarse porque el sandbox no lo permite, debes indicarlo claramente.

Nunca inventes resultados.

Nunca marques algo como aprobado si realmente no pudo comprobarse.

Entregables obligatorios

Genera los siguientes documentos:

1. PRODUCTION_ACCEPTANCE_REPORT.md
 2. DEVOPS_AUDIT_REPORT.md
 3. SECURITY_AUDIT_REPORT.md
 4. PERFORMANCE_AUDIT_REPORT.md
 5. DEPLOYMENT_READINESS_REPORT.md
 6. FINAL_GO_LIVE_CHECKLIST.md
-

Para cada archivo revisado debes indicar

Estado:

-  Aprobado
-  Mejorable
-  Crítico

Prioridad:

- P0
- P1
- P2

Impacto:

- Bajo
- Medio
- Alto
- Crítico

Descripción técnica.

Evidencia.

Riesgo.

Cómo reproducir el problema.

Cómo solucionarlo.

Nivel de confianza.

Regla final

No finalices la auditoría hasta haber revisado absolutamente todo el proyecto.

No quiero una revisión superficial.

Quiero una certificación de nivel empresarial.

Debes comportarte como un auditor externo cuyo trabajo es impedir que un sistema con fallos llegue a producción.

Tu objetivo no es demostrar que NOVSM funciona.

Tu objetivo es demostrar que NO EXISTE ningún riesgo importante antes del Go Live.

Si encuentras un solo problema crítico, el proyecto NO debe considerarse certificado para producción hasta que dicho problema sea corregido y validado nuevamente.